

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – 9

Roll No.: T017	Name: Radhika Mali
Class: TYIT	Batch: 1
Date of Assignment: 18/08/2026	Date/Time of Submission: 18/08/2026

Aim:A) Develop CRUD operations using ASP.NET Core Web API and Entity Framework

Core.

Code:

StudentController.cs:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentAPI.Data;
using StudentAPIWithEF.Models;
namespace StudentAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public StudentsController(AppDbContext context)
        {
            _context = context;
        }

        // GET ALL
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Student>>>
        GetStudents()
        {
            return await _context.Students.ToListAsync();
        }

        // GET BY ID
        [HttpGet("{id}")]
        public async Task<ActionResult<Student>>
        GetStudent(int id)
        {
            var student = await
            _context.Students.FindAsync(id);
            if (student == null)
                return NotFound();
            return Ok(student);
        }
    }
}
```

```
    }
    // INSERT
    [HttpPost]
    public async Task<ActionResult<Student>>
    AddStudent(Student student)
    {
        _context.Students.Add(student);
        await _context.SaveChangesAsync();

        return Ok(student);
    }

    // UPDATE
    [HttpPut("{id}")]
    public async Task<ActionResult> UpdateStudent(int
    id, Student student)
    {
        var data = await _context.Students.FindAsync(id);

        if (data == null)
            return NotFound();
        data.Name = student.Name;
        data.Age = student.Age;
        data.Course = student.Course;
        _context.Entry(data).State = EntityState.Modified;
        await _context.SaveChangesAsync();
        return Ok(data);
    }

    // DELETE
    [HttpDelete("{id}")]
    public async Task<ActionResult> DeleteStudent(int
    id)
    {
        var student = await
        _context.Students.FindAsync(id);
        if (student == null)
            return NotFound();
        _context.Students.Remove(student);
    }
}
```

```
await _context.SaveChangesAsync();
return Ok();
} } }
```

Aappsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\MSSQLLocalDB;Database=StudentDB;
Trusted_Connection=True;"
  }
}
```

AppDbContext.cs:

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Models;
namespace StudentAPI.Data
{
    public class AppDbContext : DbContext
    {
        public
AppDbContext(DbContextOptions<AppDbContext>
options)
        : base(options)
        {
        }
        public DbSet<Student> Students { get; set; }
    }
}
```

Program.cs:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentAPI.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
```

```
builder.Services.AddDbContext<AppDbContext>(options
=>
options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConne
ction")));
```

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

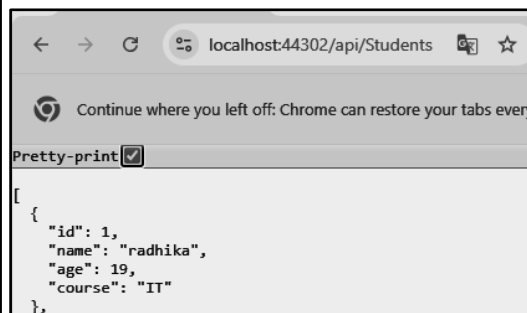
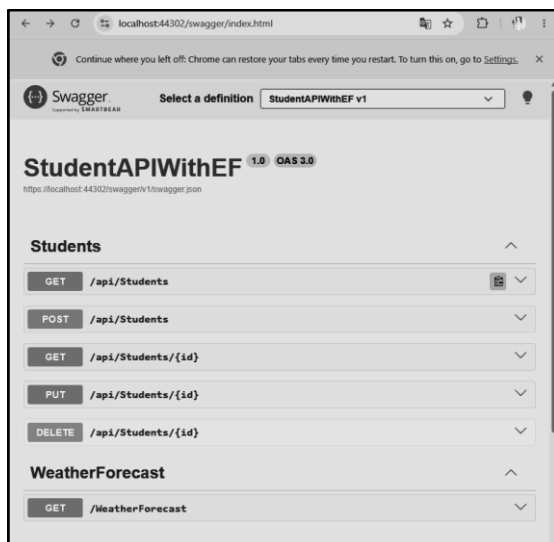
```
var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

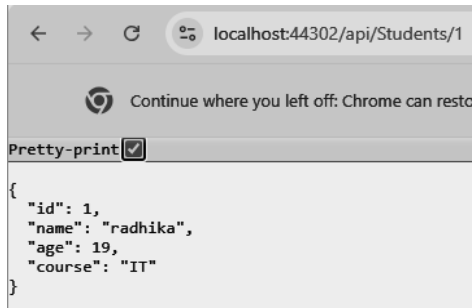
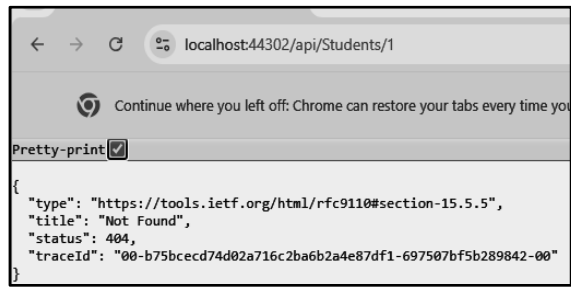
Student.cs:

```
namespace StudentAPIWithEF.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Course { get; set; }
    }
}
```

Output:

post:



Put:**Delete:**

	Id	Name	Age	Course
	2	radhika	19	IT
	NULL	NULL	NULL	NULL

Aim: B) Create an API endpoint to calculate Factorial of a Number and test it using Postman

Code:

```
using Microsoft.AspNetCore.Mvc;
namespace FactorialAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult GetFactorial(int number)
        {
            if (number < 0)
            {
                return BadRequest("Factorial not possible for negative numbers.");
            }
            long factorial = 1;
            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }
            return Ok(new
            {
                Number = number,
                Factorial = factorial
            });
        }
    }
}
```

Output: